

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 3. Python Data Types

Lecture: 20. Numeric Datatype(int & float)

Section 1: Lecture Summary

This lecture focuses on the numeric data types available in Python, specifically the integer (int) and floating-point (float) types. It begins by introducing integers, demonstrating how to declare and initialize them, and explaining their ability to store both positive and negative numbers. The instructor emphasizes Python's flexibility in handling integers of arbitrary size, unlike many other languages. Memory consumption for integers is discussed and verified using Python's sys module. The concept of immutability in integers is explained with examples showing that updating an integer variable results in a new memory allocation rather than modifying the existing one. The lecture then transitions to floating-point numbers, explaining declaration and initialization using decimal notation and scientific notation (floating point representation). Detailed explanations of mantissa and exponent components, as well as positive and negative signs for each, are provided. The instructor demonstrates these concepts through examples and practical usage in Jupyter Notebook, encouraging practice to solidify understanding.

Section 2: Key Concepts and Explanations

Integer Data Type (int)

- Declaration and Initialization

- Variables can be assigned integer values using simple assignment, e.g., `x = 101`.
- Integers can be positive or negative, e.g., `y = -15`.

- Size and Flexibility

- Python supports integers of arbitrary size without a fixed maximum number of digits (unlike C/C++/Java).

- Examples show integers with 3, 6, 9, 16 digits and beyond.
- Memory Consumption
 - The amount of memory consumed depends on the size of the integer.
 - Python's `sys.getsizeof()` function reveals, e.g., 28 bytes for smaller integers and 32 bytes for larger ones.
- Immutability
 - Integers are immutable; changing a value creates a new integer object in memory.
 - Use of Python's built-in `id()` function demonstrates different memory addresses before and after reassignment.
 - The original object's memory is reclaimed by Python's garbage collector.

Floating-Point Data Type (float)

- Declaration and Initialization
 - Float values are declared using decimal points, e.g., `a = 101.0` or `b = 0.075`.
 - Using scientific notation with an "E" or "e" also defines floats, e.g., `c = 2.5e-2`.
- Floating Point Representation (Scientific Notation)
 - Combines a mantissa (significant digits) and an exponent (power of 10).
 - Examples:
 - $12500 = 125 \times 10^2 \rightarrow$ represented as `125E2`.
 - $23.45 = 2345 \times 10^{-2} \rightarrow$ represented as `2345E-2`.
 - The decimal point "floats" to adjust magnitude.
- Positive and Negative Values
 - Both mantissa and exponent can be positive or negative.
 - Examples include positive floats, negative floats, and floats with negative exponents.

Using Jupyter Notebook for Practice

- Demonstrations include declaring variables, printing values, and verifying concepts with actual code.

- Encouraged to practice declaring and manipulating numeric types interactively.

Section 3: Example Code and Use Cases

Integer declarations

Printing integer variables

```
x = 101          # positive integer
y = -15          # negative integer
large_num = 1234567890123456 # large integer allowed in Python

print(x)
print(y)
print(large_num)
```

Demonstrates integer declaration, both positive and negative, including large integers.

```
import sys

x = 101
y = 1234567890123456

print(sys.getsizeof(x)) # Output: 28 (size in bytes)
print(sys.getsizeof(y)) # Output: 32 (larger due to size of data)
```

Shows how to check memory consumption of integers depending on their size.

```
x = 101
print(id(x))    # Print memory address of x

x = 205
print(id(x))    # New memory address after assigning new value
```

Illustrates immutability: changing the integer value creates a new object with a different memory location.

Float declarations

```
a = 101.0      # float with decimal point
b = 0.075      # float starting with decimal
c = -1.75      # negative float
d = 2.5e2       # float in scientific notation (250.0)
e = -3.5E-2     # negative mantissa and exponent (-0.035)

print(a, b, c, d, e)
```

Shows float declarations including decimal and scientific notation forms with positive and negative values.

Section 4: Key Takeaways

- Python numeric types include integer (int), float, Boolean, and complex; this lecture covers int and float.
- Integers in Python support any number of digits and can be positive or negative.
- Memory use for integers depends on their size; verified via `sys.getsizeof()`.
- Integers are immutable; changing their value results in new memory allocation.
- Floats are declared using decimal notation or scientific notation (`E`` or `e``).
- Floating point representation consists of a mantissa and an exponent, explaining how floats encode numbers.
- Both mantissa and exponent can be positive or negative in floats.
- Jupyter Notebook is a useful tool for practicing and verifying numeric types and concepts in Python.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.